ChessOTB.club — Game Recorder: Build Pack v2.0

Gap-Closed Edition | March 2026 | Supersedes v1.0

Foreword: What Changed and Why

Version 1.0 of this build pack was a solid foundation — correct architecture, realistic scope, and a workable engineering roadmap. Three gaps needed closing before construction begins. First, the computer vision strategy was too shallow: delegating board reconstruction to a generic vision API ignores the specific failure modes of casual OTB environments (hands, drinks, reflections, non-standard sets). Second, the timestamp schema was deferred entirely, which would make video synchronization a painful retrofit rather than a clean addition. Third, the social and growth layer was explicitly excluded, which is strategically correct for MVP but must be designed into the data model now so it does not require a schema migration later.

This document closes all three gaps and adds three new features — the Game Highlight Generator, the OTB Accuracy Rating, and Opening Detection — that together define the "Strava for OTB chess" growth loop.

Part 1 — Gap 1 Closed: Two-Stage Computer Vision Architecture

Why Generic Vision APIs Fail for OTB Chess

A casual OTB environment is not a controlled studio. The camera is propped on a water bottle or a stack of books. Hands reach across the board every 30 seconds. A coffee cup sits on the h-file. The opponent's phone is face-down on the a1 corner. Pieces are knocked over and replaced. Lighting is uneven, often from a single overhead bulb creating harsh shadows on one half of the board.

Generic vision APIs — OpenAI Vision, Google Vision, chess-snapshot-api — treat each frame as an independent image classification problem. They have no temporal context, no understanding of chess rules, and no ability to distinguish a hand temporarily covering pieces from a permanent board state change. Their failure rate in real OTB conditions is high enough to make them unreliable as a primary reconstruction method.

The correct architecture is a **two-stage pipeline** built on purpose-specific tools: OpenCV for board geometry and YOLOv8 for piece classification.

Stage 1 — Board Detection (OpenCV)

The first stage locates the chessboard in the frame and normalizes the perspective. This is a classical computer vision problem that OpenCV solves reliably without machine learning.

Step 1.1 — Canny Edge Detection. Apply a Canny edge detector to the grayscale frame. The chessboard's alternating light and dark squares produce strong, consistent edges that survive most lighting conditions.

Step 1.2 — Hough Line Transform. Apply a probabilistic Hough line transform to detect the grid lines of the board. A standard 8×8 board produces a characteristic pattern of 9 horizontal and 9 vertical lines. Filter for lines that form near-right-angle intersections at regular intervals.

Step 1.3 — Corner Detection. Identify the four outermost intersection points as the board corners. Use Harris corner detection or the Shi-Tomasi algorithm to refine the corner coordinates to sub-pixel accuracy.

Step 1.4 — Homographic Perspective Normalization. Compute the homography matrix that maps the four detected corners to a canonical 400×400 pixel top-down view. Apply `cv2.warpPerspective()` to produce a normalized board image. This step eliminates the camera angle as a variable — all downstream processing works on a consistent top-down view regardless of where the phone is positioned.

Step 1.5 — Occlusion Detection. Before passing the normalized frame to Stage 2, check for occlusion. A hand or object covering the board disrupts the edge pattern. If the Hough line detection fails to find the expected grid pattern, mark the frame as `occluded` and skip it. Occluded frames are not passed to the piece classifier — they are simply dropped from the reconstruction timeline.

Robustness note: Board detection using OpenCV Hough lines achieves greater than 95% accuracy on standard sets under reasonable lighting conditions, as documented in academic literature on chessboard digitization. The primary failure mode is non-standard

sets (e.g., travel sets with round pieces, novelty sets with unusual colors). The setup screen should warn users that non-standard sets may reduce reconstruction accuracy.

Stage 2 — Piece Classification (YOLOv8)

The second stage identifies which piece occupies each square of the normalized board. This is where machine learning is genuinely necessary — the visual difference between a knight and a bishop, or a queen and a king, requires learned feature recognition.

Model architecture: YOLOv8-nano (the smallest variant, ~3.2M parameters) fine-tuned on a chess piece dataset. The nano variant is chosen for inference speed — it runs at approximately 60 FPS on a modern CPU, which is more than sufficient for processing 1 FPS video frames. Academic work on chess piece detection with YOLOv8 reports accuracy above 95% on standard sets under controlled conditions.

Training data: The `chess-yolov8` dataset (GitHub: shainisan/chess-yolov8) provides a starting point with labeled images of chess pieces. Augmentation with rotation, brightness variation, and partial occlusion is essential to handle real-world conditions. The model should be trained to classify 13 classes: empty square, white/black pawn, knight, bishop, rook, queen, and king.

Inference approach: Rather than running the full YOLOv8 detector on the entire board image, divide the normalized 400×400 board into a 8×8 grid of 50×50 pixel squares and run a lightweight classifier on each square independently. This reduces the inference problem from object detection (finding and classifying) to pure classification (what is on this square), which is faster and more accurate for this specific use case.

Deployment: The model runs server-side as a Python microservice. The Node.js backend sends normalized frame images to the Python service via an internal HTTP endpoint. The Python service returns a JSON array of 64 piece classifications with confidence scores. This separation keeps the Node.js server clean and allows the CV pipeline to be scaled, updated, or replaced independently.

```
Node.js Backend
    │
    ├── POST /internal/classify-frame
    │       { frameKey: "recordings/abc/frames/0045.jpg" }
    │
    └── Python CV Service (FastAPI)
            ├── Load frame from S3
            ├── Normalize perspective (OpenCV)
            ├── Classify 64 squares (YOLOv8-nano)
            └── Return { squares: [...], confidence: 0.94, occluded: false }
```

Stage 3 — Move Inference (chess.js)

The output of Stage 2 is a sequence of FEN strings, one per processed frame. Move inference compares consecutive FEN strings to detect board state changes.

Change detection: When two consecutive FENs differ, the system identifies which squares changed. A move involves exactly two square changes (source and destination) for a normal move, or three changes for a capture (source empties, destination changes piece type), or four changes for castling.

Legal move validation: Every inferred move is validated against chess.js's legal move generator for the current position. If the inferred move is illegal, it is flagged for manual correction regardless of the classifier's confidence score. This is a critical safety net — it catches classification errors before they corrupt the PGN.

Confidence scoring: Each inferred move receives a confidence score computed from the minimum classifier confidence across all changed squares. A move where every changed square was classified with >90% confidence receives a high confidence score. A move where any square was below 70% confidence is flagged for manual correction.

Temporal deduplication: Multiple consecutive frames may show the same board state (e.g., 30 frames between moves). The system deduplicates by only recording a new move when the board state changes. The timestamp of the first frame showing the new state is recorded as the move timestamp.

Failure Mode Handling

The two-stage pipeline has well-defined failure modes, each with a specific recovery path.

Failure Mode	Cause	Recovery
Board not detected	Non-standard set, extreme angle, poor lighting	Skip frame, continue with next frame
Occluded frame	Hand, drink, or object covering board	Skip frame, mark timestamp gap
Illegal move inferred	Classifier error or ambiguous position	Flag for manual correction
Confidence below threshold	Unclear piece (e.g., bishop vs queen in shadow)	Flag for manual correction
Too many consecutive skipped frames	Camera moved, board out of frame	Pause reconstruction, resume when board re-detected
Reconstruction confidence below 80% overall	Poor video quality throughout	Fall back to manual PGN entry, offer partial reconstruction as starting point

---

Part 2 — Gap 2 Closed: Timestamp Schema Design

Why This Must Be Designed Now

The v1.0 spec deferred video-to-board synchronization to post-MVP. That decision is correct for the user-facing feature — synced playback is a Phase 2 feature. But the data must be stored now. Retrofitting a timestamp column into a production database after thousands of games have been analyzed requires a migration, backfill logic, and a period of inconsistent data. The cost of storing timestamps from day one is zero. The cost of not storing them is a painful migration.

Revised `MoveAnalysis` Schema

The `move_analyses` table gains three new columns: `timestamp_ms`, `timestamp_confidence`, and `frame_key`.

```sql
ALTER TABLE move_analyses ADD COLUMN timestamp_ms INTEGER;
-- Video timestamp in milliseconds from the start of the recording.
-- NULL if the game was entered via manual PGN (no video).
-- Populated by the vision pipeline when a move is detected.

ALTER TABLE move_analyses ADD COLUMN timestamp_confidence REAL;
-- Confidence score for the timestamp mapping (0.0–1.0).
-- High confidence = the frame was clear and the move was unambiguous.
-- Low confidence = the timestamp was interpolated from surrounding frames.
-- NULL if no video.

ALTER TABLE move_analyses ADD COLUMN frame_key TEXT;
-- S3 key of the video frame where this move was detected.
-- Example: "recordings/abc123/frames/0347.jpg"
-- Used to show the video still in the correction flow.
-- NULL if no video.
```

Revised `ProcessedGame` Schema

The `processed_games` table gains a `move_timestamps` JSON column that stores the complete timestamp array for fast retrieval without joining `move_analyses`.

```sql
ALTER TABLE processed_games ADD COLUMN move_timestamps JSONB;
-- Array of { moveNumber, color, timestampMs, confidence } objects.
-- Example: [{ moveNumber: 1, color: "w", timestampMs: 4200, confidence: 0.97 },
...]
-- Redundant with move_analyses.timestamp_ms but optimized for the sync player.
-- NULL if no video.
```

Timestamp Population Strategy

Timestamps are populated at three points in the pipeline, each with a different confidence level.

High confidence (0.85–1.0): The vision pipeline detects a clear board state change in a non-occluded frame. The timestamp is the frame's position in the video (frame number ÷ frame rate × 1000 ms).

Medium confidence (0.50–0.84): The board state change was detected but the surrounding frames were partially occluded. The timestamp is interpolated from the nearest high-confidence timestamps on either side.

Low confidence (0.0–0.49): The move was flagged for manual correction. After the user confirms the move, the timestamp is assigned based on the video frame shown in the correction card. This is the most accurate timestamp possible for uncertain moves — the user is literally looking at the frame.

No video (NULL): The game was entered via manual PGN. All timestamp fields are NULL. The sync player is not shown on the analysis page.

How the Sync Player Uses This Data

When the analysis page loads and `move_timestamps` is not NULL, the video player is shown. The sync system works as follows: as the video plays, the current playback position (in ms) is compared against the `move_timestamps` array. When the playback position crosses a move's `timestampMs`, the board advances to that position. The tolerance window is ±1.5 seconds for high-confidence timestamps and ±3 seconds for low-confidence timestamps.

The reverse direction — clicking a move to seek the video — uses the `timestampMs` directly. The video seeks to `timestampMs - 2000` (2 seconds before the move) to give context. This is the "Watch this move" experience.

Part 3 — Gap 3 Closed: Social and Growth Layer Design

The Strategic Framing

The "Strava for OTB chess" vision requires a social layer, but the social layer must not ship before the core analysis experience is solid. The v1.0 spec was right to exclude sharing from MVP. The revision here is not to add sharing to MVP — it is to **design the data model for sharing now** so that adding it post-MVP is a feature addition, not a schema migration.

Data Model Additions for Future Sharing

Two columns are added to `processed_games` to support future public sharing without requiring a schema change.

```sql
ALTER TABLE processed_games ADD COLUMN is_public BOOLEAN DEFAULT FALSE;
-- When true, the game is accessible via its public share URL.
-- Default false — all games are private until the user explicitly shares.

ALTER TABLE processed_games ADD COLUMN share_token TEXT UNIQUE;
-- A random 12-character token used in the public share URL.
-- Example: /game/share/a3k9mz2p1qr8
-- Generated when the user first enables sharing.
-- NULL until sharing is enabled.
```

The public share URL pattern is `/game/share/:shareToken`. This route does not require authentication. It shows a read-only version of the analysis page with the player names, move list, eval bar, and accuracy panel — but without the video (video URLs are presigned S3 URLs that expire).

The Three Growth Features

Feature A — Game Highlight Generator

The Game Highlight Generator produces a shareable static image (not a video clip) showing the most dramatic moment of the game. A video clip is technically possible via Canvas API + MediaRecorder, but it adds significant complexity and file size. A static image is shareable on every platform, loads instantly, and can be generated server-side without browser APIs.

What it generates: A 1080×1080 pixel PNG (Instagram-square format) containing: the board position at the key moment, the eval bar showing the swing, the move notation and classification, the player names and result, and the ChessOTB.club branding. The image is generated server-side using `node-canvas` (the Node.js Canvas API) and stored in S3.

Trigger: The highlight is generated automatically after analysis completes. The system identifies the single move with the largest centipawn swing (the “moment of the game”) and generates the highlight image for that move. The user can also tap any move in the move list to generate a highlight for that specific moment.

Share flow: The user taps “Share Highlight” on the analysis page. The native Web Share API is invoked with the PNG file (already built into the platform from the Report page share feature). On platforms that do not support file sharing, the image is shown in a modal with a “Download” button and a “Copy Link” button that copies the public game URL.

Copy for the share card:

> *"Move 23 — the moment the game turned. White missed mate in 3. #ChessOTB"*

The copy is generated dynamically based on the classification: blunders get "missed mate in N" or "dropped a piece", brilliant moves get "found the only winning move", and decisive evaluation swings get the centipawn delta.

Feature B — OTB Accuracy Rating

The OTB Accuracy Rating is a single number that summarizes a player's performance in a game. It is displayed prominently on the analysis page and on the game card in the My Games list.

Calculation method: The accuracy percentage is computed using the same formula Lichess uses, which is based on win probability rather than raw centipawn loss. For each move, the win probability before the move (`wp_before`) and after the move (`wp_after`) are compared. The accuracy for that move is `max(0, 103.1668 * exp(-0.04354 * (wp_before - wp_after) * 100) - 3.1669)`. The game accuracy is the average across all moves. This formula produces a number between 0 and 100 that is more meaningful than raw centipawn loss because it accounts for the game phase — a 50 cp loss in a dead-equal endgame is less significant than a 50 cp loss in a sharp tactical position.

Display format on the analysis page:

```
White: Magnus C.          Black: Hikaru N.
OTB Accuracy: 91%         OTB Accuracy: 78%
Blunders: 0               Blunders: 2
Mistakes: 1               Mistakes: 5
Best move streak: 12      Best move streak: 7
```

Display format on the game card (My Games list):

```
[Board thumbnail]  Kasparov vs Karpov  1-0
                   Sicilian Defense · 42 moves
                   Your accuracy: 84%  ·  Mar 7, 2026
```

Long-term value: After a player has analyzed 10+ games, the My Games list shows an accuracy trend line. After 20+ games, the Profile page shows their "OTB Accuracy Rating" — a rolling 10-game average — alongside their chess.com ELO. This is the personal stat that brings players back to the platform.

Feature C — Opening Detection

Opening detection adds the opening name and ECO code to every analyzed game. It is a pure data enrichment step that requires no additional API calls — it runs entirely from a local lookup table.

Implementation: The `chess-opening-book-reader` npm package provides a lookup table of 3,000+ openings indexed by the first 10–15 moves of the game. After the PGN is submitted and validated, the server iterates through the move list and compares the position after each move against the opening book. The longest matching sequence determines the opening name and ECO code.

Fallback: If no opening is found (e.g., the game starts with an unusual first move), the opening is recorded as "Irregular Opening" with ECO code "A00". This is always populated — the opening field is never NULL.

Display on the analysis page: The opening name and ECO code appear in the Game Info card: *"Sicilian Defense: Najdorf Variation (B90)"*. The ECO code is a hyperlink to the Lichess opening explorer for that code, allowing the player to study the opening further.

Display on the game card: The opening name appears as a subtitle on the game card in the My Games list, giving each game a distinct identity at a glance.

Part 4 — Revised Engineering Task List

The task list from v1.0 is revised to incorporate the three gap-closing additions. New tasks are marked with **[NEW]**. Revised tasks are marked with **[REV]**.

Sprint 1 — Core Analysis Flow (Week 1)

Unchanged from v1.0 with two additions.

#	Task	Complexity	Notes
1.1	Refactor status to 12-state machine	S	—
1.2	Progressive analysis loading	M	—
1.3	Horizontal eval bar on all breakpoints	S	—
1.4	Sticky bottom nav strip on mobile	S	—
1.5	Touch swipe navigation	M	—
1.6	Move info strip (classification + best move)	S	—
1.7	Analysis polling with exponential backoff	S	—
1.8	"Analyzing…" banner during `analyzing` state	S	—
1.9	My Games list for returning users	M	—
1.10	`GET /api/recordings` endpoint	S	—
1.11	**[NEW] Opening detection via chess-opening-book-reader**	**S**	**Run server-side after PGN submission**
1.12	**[NEW] Add `is_public`, `share_token` columns to `processed_games`**	**S**	**Schema change, no UI yet**
1.13	**[NEW] Add `timestamp_ms`, `timestamp_confidence`, `frame_key` to `move_analyses`**	**S**	**Schema change, no UI yet**
1.14	**[NEW] Add `move_timestamps` JSONB column to `processed_games`**	**S**	**Schema change, no UI yet**

Sprint 2 — Recording Flow (Week 2)

Unchanged from v1.0 with one revision.

#	Task	Complexity	Notes
2.1	`/record/camera` page with live camera preview	M	—
2.2	Board framing overlay	S	—
2.3	Wake lock implementation	S	—
2.4	MediaRecorder with codec fallback	M	—
2.5	Recording in-progress screen	S	—
2.6	Chunked video upload to S3	L	—
2.7	Processing status screen	M	—
2.8	Battery level warning	S	—
2.9	`POST /api/recordings/:id/upload` endpoint	M	—
2.10	Video player on analysis page	M	—

Sprint 3 — Correction Flow + Highlight Generator (Week 3)

The correction flow tasks are unchanged. The Highlight Generator is added.

#	Task	Complexity	Notes
3.1	`CorrectionCard` component	L	—
3.2	Correction batch flow	M	—
3.3	“None of these” drag-to-move fallback	L	—
3.4	⚠ icon for skipped corrections in move list	S	—
3.5	Partial re-analysis after corrections	M	—
3.6	`POST /api/recordings/:id/corrections` endpoint	S	—
3.7	Signed-out state on `/record`	S	—
3.8	“Enter PGN Instead” escape hatch	S	—
3.9	Error recovery screens	M	—
3.10	My Games section on Profile page	M	—
3.11	**[NEW] Server-side highlight image generator (node-canvas)**	**L**	**1080×1080 PNG with board, eval, move, branding**
3.12	**[NEW] “Share Highlight” button on analysis page**	**S**	**Uses existing Web Share API infrastructure**
3.13	**[NEW] Auto-generate highlight for top eval swing after analysis completes**	**M**	**Triggered by `complete` status transition**
3.14	**[NEW] OTB Accuracy Rating display on analysis page**	**S**	**Accuracy % already computed — display refinement**
3.15	**[NEW] OTB Accuracy Rating on game card in My Games list**	**S**	**Requires accuracy stored on `processed_games`**

Sprint 4 — Two-Stage CV Pipeline (Week 4)

The v1.0 Sprint 4 is replaced entirely with the two-stage architecture.

#	Task	Complexity	Notes
4.1	[REV] Set up Python FastAPI microservice for CV pipeline	M	Separate process, internal HTTP endpoint
4.2	[REV] Implement OpenCV board detection (Canny + Hough + homography)	L	Python, runs on extracted frames
4.3	[REV] Implement occlusion detection (skip frames where board grid not found)	M	Part of Stage 1
4.4	[REV] Fine-tune YOLOv8-nano on chess piece dataset	XL	Requires GPU for training; use pre-trained weights as starting point
4.5	[REV] Implement 64-square classifier (crop + classify each square)	L	Faster and more accurate than full-board YOLO detection
4.6	[REV] Implement move inference from FEN sequence (chess.js validation)	L	Flags illegal moves for correction
4.7	[REV] Implement confidence scoring per move	M	Drives correction flow threshold
4.8	[REV] Populate `timestamp_ms` and `frame_key` during move inference	M	Gap 2 fix — store timestamps as moves are detected
4.9	[REV] Integrate Python CV service with Node.js backend	M	Internal HTTP call from recordings.ts
4.10	[REV] Add “Beta” badge and accuracy disclaimer to camera option	S	Honest about CV limitations

---

Part 5 — Revised Component Inventory

New Components Added in v2.0

Component	File	Purpose	Sprint
`GameHighlightCard`	`components/GameHighlightCard.tsx`	Renders the shareable highlight image layout (board + eval + move + branding)	3
`OpeningBadge`	`components/OpeningBadge.tsx`	Displays ECO code + opening name as a styled badge	1
`AccuracyRatingBadge`	`components/AccuracyRatingBadge.tsx`	Compact accuracy % display for game cards	3
`ShareHighlightButton`	`components/ShareHighlightButton.tsx`	Triggers highlight generation + Web Share API	3
`VideoSyncPlayer`	`components/VideoSyncPlayer.tsx`	Video player that accepts `moveTimestamps` prop for future sync	2

New API Endpoints Added in v2.0

Method	Path	Purpose	Sprint
`POST`	`/api/games/:id/highlight`	Generate highlight image for a specific move	3
`GET`	`/api/games/:id/highlight/:moveNumber`	Get presigned S3 URL for a highlight image	3
`POST`	`/api/games/:id/share`	Enable public sharing, generate `share_token`	Post-MVP
`GET`	`/game/share/:shareToken`	Public read-only analysis page	Post-MVP
`POST`	`/internal/classify-frame`	Python CV service endpoint (internal only)	4

New Server-Side Libraries

Library	Purpose	Language
`node-canvas`	Server-side PNG generation for highlight cards	Node.js
`chess-opening-book-reader`	ECO opening detection from move sequence	Node.js
`opencv-python`	Board detection (Canny, Hough, homography)	Python
`ultralytics` (YOLOv8)	Piece classification	Python
`fastapi`	Python CV microservice HTTP server	Python
`chess` (python-chess)	FEN validation in Python pipeline	Python

Part 6 — Revised Launch Checklist

The v1.0 checklist is retained in full. The following items are added.

Gap-Closing Requirements

- [ ] `move_analyses` table has `timestamp_ms`, `timestamp_confidence`, `frame_key` columns (NULL for manual PGN games)
- [ ] `processed_games` table has `move_timestamps` JSONB column (NULL for manual PGN games)
- [ ] `processed_games` table has `is_public` and `share_token` columns (both NULL/false by default)
- [ ] Opening detection runs for every submitted PGN; `openingName` and `openingEco` are never NULL
- [ ] Opening name and ECO code are displayed on the analysis page Game Info card
- [ ] Opening name is displayed on the game card in the My Games list

Highlight Generator Requirements

- [ ] Highlight image is auto-generated for the top eval swing after analysis completes
- [ ] Highlight image is stored in S3 under `games/:gameId/highlights/:moveNumber.png`
- [ ] "Share Highlight" button appears on the analysis page after analysis completes
- [ ] Share text is dynamically generated based on move classification (blunder, brilliant, etc.)
- [ ] Highlight image renders correctly at 1080×1080 pixels with all text legible
- [ ] Highlight generation does not block the analysis page from loading

OTB Accuracy Rating Requirements

- [ ] Accuracy percentage is computed using the win-probability formula (not raw centipawn average)
- [ ] Accuracy is stored on `processed_games` as `whiteAccuracy` and `blackAccuracy` (0–100)
- [ ] Accuracy is displayed prominently on the analysis page for both players
- [ ] Accuracy is displayed on the game card in the My Games list
- [ ] Best move streak is computed and displayed (longest consecutive sequence of best/good moves)

CV Pipeline Requirements (Sprint 4)

- [ ] Python CV microservice starts and responds to `/internal/classify-frame` requests
- [ ] Board detection correctly identifies board corners in at least 80% of test frames
- [ ] Occlusion detection correctly skips frames where a hand covers the board
- [ ] Move inference produces a valid PGN for at least 70% of test games without manual correction
- [ ] Timestamp is stored for every detected move with a confidence score
- [ ] Camera option on `/record` page displays “Beta” badge with tooltip explaining limitations

---

Part 7 — The Growth Loop

The three new features — Opening Detection, OTB Accuracy Rating, and Game Highlight Generator — are not independent additions. They form a reinforcing growth loop that is the long-term strategic value of the platform.

A player finishes a game at their club. They open ChessOTB.club, paste their moves, and within 60 seconds they have: their OTB Accuracy Rating for the game (a number they want to improve), the opening they played (a name they can study), and a shareable highlight card showing the moment the game turned. They share the highlight card on their chess group chat. Three people in the group click the link. Two of them sign up to analyze their own games.

This is the “Strava for OTB chess” loop. Strava did not grow because it had the best GPS tracking — it grew because every run produced a shareable artifact (the activity card) that brought new users in. The Game Highlight Card is that artifact for ChessOTB.club.

The OTB Accuracy Rating is the retention mechanic. Players return not just to analyze individual games but to track whether their accuracy is improving over time. After 10 games, the My Games list shows a trend. After 20 games, the Profile page shows a rolling accuracy rating. This is the personal stat that turns a one-time user into a regular.

Opening Detection is the depth mechanic. Knowing you played the Sicilian Najdorf gives you something to study. A future “Opening Repertoire” page — showing which openings you play most, your accuracy in each, and your win rate — is a natural extension that requires no new data collection, only new presentation of data already being stored.

Appendix — Revised Data Model (Complete)

```
recording_sessions
  id                    TEXT PRIMARY KEY
  userId                TEXT NOT NULL
  tournamentId          TEXT (nullable)
  status                TEXT NOT NULL  -- 12-state machine
  videoKey              TEXT (nullable)  -- S3 key
  createdAt             TIMESTAMP
  updatedAt             TIMESTAMP

processed_games
  id                    TEXT PRIMARY KEY
  sessionId             TEXT NOT NULL
  pgn                   TEXT NOT NULL
  moveTimestamps        JSONB (nullable)  -- [{ moveNumber, color, timestampMs,
confidence }]
  openingName           TEXT NOT NULL DEFAULT 'Irregular Opening'
  openingEco            TEXT NOT NULL DEFAULT 'A00'
  totalMoves            INTEGER NOT NULL
  whitePlayer           TEXT
  blackPlayer           TEXT
  result                TEXT  -- '1-0', '0-1', '1/2-1/2'
  whiteAccuracy         REAL (nullable)  -- 0–100
  blackAccuracy         REAL (nullable)  -- 0–100
  is_public             BOOLEAN DEFAULT FALSE
  share_token           TEXT UNIQUE (nullable)
  createdAt             TIMESTAMP

move_analyses
  id                    TEXT PRIMARY KEY
  gameId                TEXT NOT NULL
  moveNumber            INTEGER NOT NULL
  color                 TEXT NOT NULL  -- 'w' or 'b'
  san                   TEXT NOT NULL
  fen                   TEXT NOT NULL
  eval                  INTEGER (nullable)  -- centipawns
  bestMove              TEXT (nullable)  -- SAN
  classification        TEXT (nullable)  -- best/good/inaccuracy/mistake/blunder
  winChance             REAL (nullable)  -- 0.0–1.0
  continuation          TEXT (nullable)  -- JSON array of SAN moves
  timestamp_ms          INTEGER (nullable)  -- milliseconds from video start
  timestamp_confidence REAL (nullable)  -- 0.0–1.0
  frame_key             TEXT (nullable)  -- S3 key of source frame

correction_entries
  id                    TEXT PRIMARY KEY
```

```
gameId              TEXT NOT NULL
moveNumber          INTEGER NOT NULL
candidateMoves      JSONB NOT NULL  -- [{ san, fen, confidence }]
chosenMove          TEXT (nullable)
confidence          REAL (nullable)
skipped             BOOLEAN DEFAULT FALSE
frameKey            TEXT (nullable)  -- S3 key shown in correction card
timestampMs         INTEGER (nullable)
```